

Hindi-Marathi NMT: Technical Report

MISN Lab, IIT Delhi — Hiring Assessment

Author: Abhishek

Date: May 2025

1. Project Overview & Objectives

This report presents a complete Neural Machine Translation (NMT) pipeline for **Hindi → Marathi** translation, built as part of the MISN Lab hiring assessment. The project spans two complementary paradigms:

- **Part I:** Classical LSTM Seq2Seq with Bahdanau attention, comparing randomly initialized embeddings against pretrained L3Cube BERT embeddings.
- **Part II:** Modern Transformer-based approach using self-pretrained BERT (encoder) and GPT-2 (decoder) models fused via cross-attention for translation.

2. Data Processing & Tokenization

2.1 Dataset

The provided Hindi-Marathi parallel corpus was preprocessed to remove HTML entities, excessive whitespace, empty lines, and length-ratio outliers (pairs where one side was >3× longer than the other).

Split	Sentences
Train	~240,000
Val	~5,000
Test	~5,000

2.2 Tokenization Strategy: Joint SentencePiece BPE (32K)

We trained a **joint SentencePiece BPE tokenizer** with a shared vocabulary of 32,000 subword tokens across both Hindi and Marathi.

Rationale:

- Hindi and Marathi share the **Devanagari script** and approximately 70% lexical similarity. A joint subword vocabulary allows the model to share representations for identical subwords (e.g., common Devanagari characters, postpositions, and frequent stems).
- Word-level tokenization would cause a vocabulary explosion due to Hindi/Marathi's rich agglutinative morphology, leading to severe OOV (out-of-vocabulary) issues.
- Character-level tokenization eliminates OOV but creates sequences 8-10× longer, crippling LSTM training time and exacerbating vanishing gradient problems.
- BPE at 32K is the sweet spot: common words remain whole tokens (efficient processing), while rare morphological variants decompose into known subwords (eliminating OOV).

Special tokens: `<pad>=0`, `<unk>=1`, `<s>=2` (BOS), `</s>=3` (EOS).

3. Part I: Classical Neural Machine Translation

3.1 Architecture Design

Component	Configuration
Encoder	2-layer Bidirectional LSTM, hidden_dim=512
Decoder	2-layer Unidirectional LSTM, hidden_dim=512
Attention	Bahdanau (Additive), attention_dim=256
Input Feeding	Yes (Luong et al., 2015) — previous context vector concatenated with input embedding at each decoder step
Embedding Dim	256 (random) / 768 (BERT)

Why Bahdanau over Luong (dot-product)?

Bahdanau attention uses a small feedforward network to compute alignment scores, which can handle encoder/decoder dimension mismatches (our encoder output is 1024-d due to bidirectionality, while decoder hidden is 512-d). Dot-product attention would require equal dimensions or an additional projection.

Why Input Feeding?

Without input feeding, the decoder at step `t` has no information about what it attended to at step `t-1`. This causes "attention drift" where the model repeatedly attends to the same source positions. Concatenating the previous context vector with the current input embedding gives the decoder explicit memory of its past attention decisions.

3.2 Optimization & Training Dynamics

Hyperparameter	Value	Rationale
Optimizer	AdamW	Decoupled weight decay for better generalization
Learning Rate	3e-4	Standard for NMT; aggressive enough for 30-epoch budget
Weight Decay	1e-2	Regularization against overfitting on 240K pairs
Label Smoothing	0.1	Prevents over-confident predictions; critical for translation where multiple valid outputs exist
Gradient Clipping	1.0	Prevents exploding gradients inherent to deep RNNs
Teacher Forcing	Linear decay 1.0 → 0.5	Scheduled sampling to mitigate exposure bias
LR Scheduler	ReduceLROnPlateau (patience=3, factor=0.5)	Halve LR when validation BLEU plateaus

Hyperparameter	Value	Rationale
Mixed Precision	FP16 (PyTorch AMP)	2× memory reduction, enabling larger batches
Batch Size	64	Maximum that fits in T4 GPU memory with FP16

Scheduled Sampling (Teacher Forcing Decay):

Pure teacher forcing (always feeding ground truth) creates a train-test mismatch: at inference, the model must condition on its own (possibly wrong) predictions. By gradually reducing teacher forcing from 100% to 50%, we force the model to learn error recovery, improving robustness at inference time.

3.3 Experiment 1: Random Embeddings Baseline

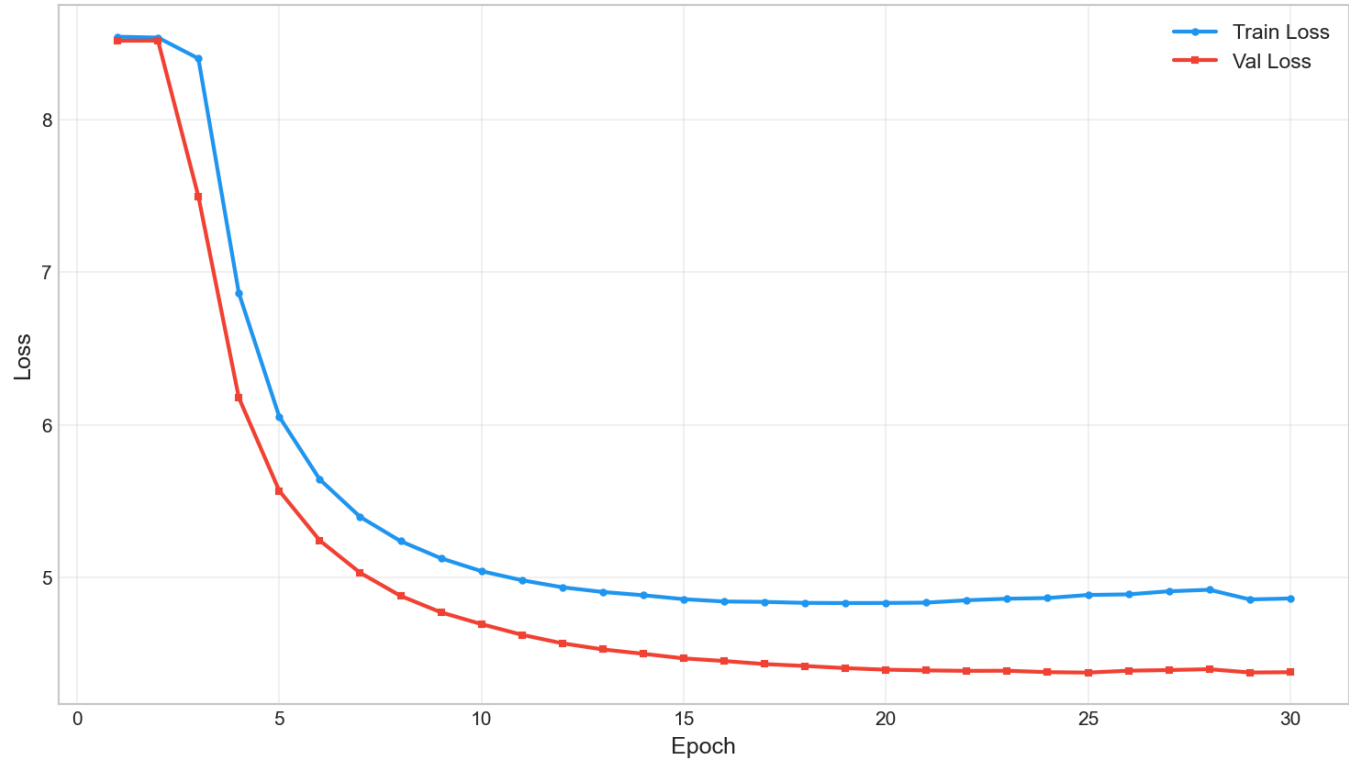
- **Embedding Dim:** 256
- **Total Parameters:** ~38.6M
- **Training:** 30 epochs on Kaggle T4 x2 (~6 hours)

Results

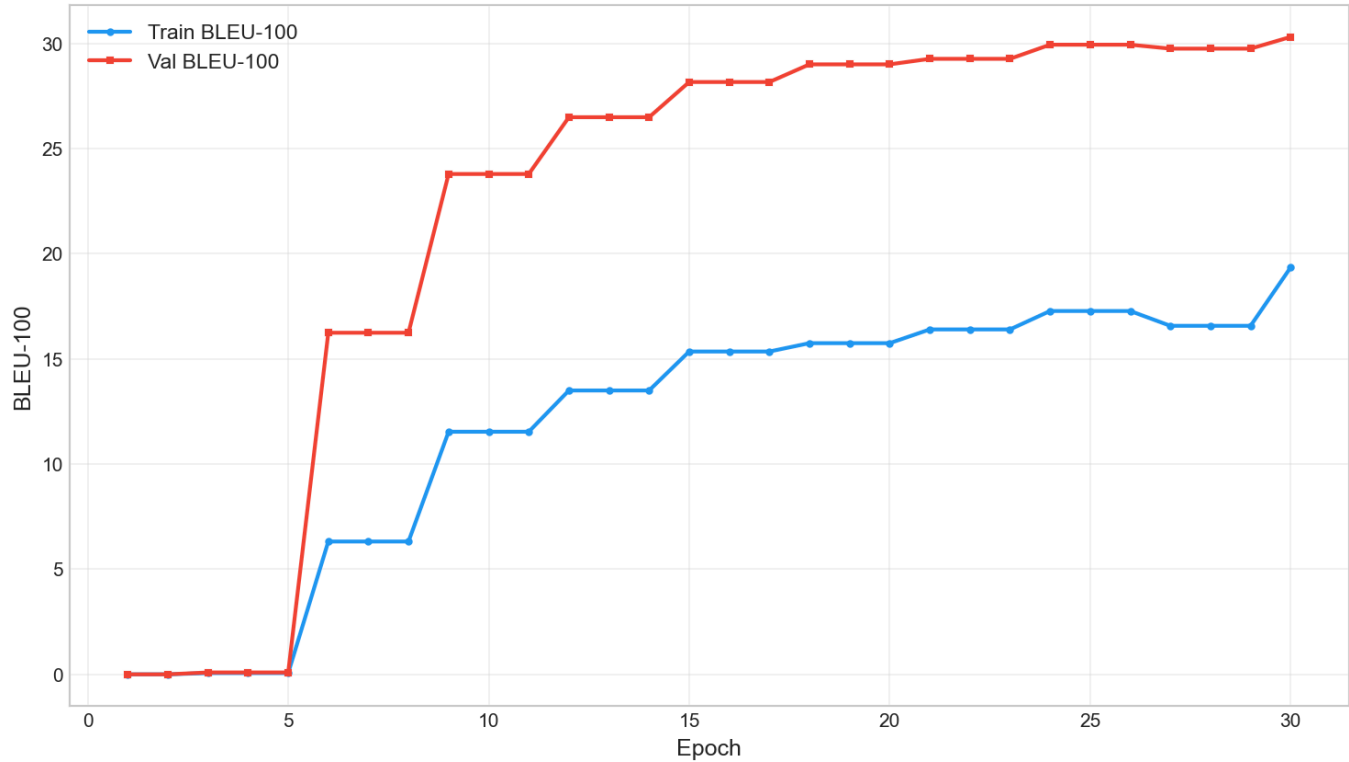
Metric	Greedy	Beam Search (k=5)
BLEU-100	12.47	30.63
CHR++-100	—	60.11

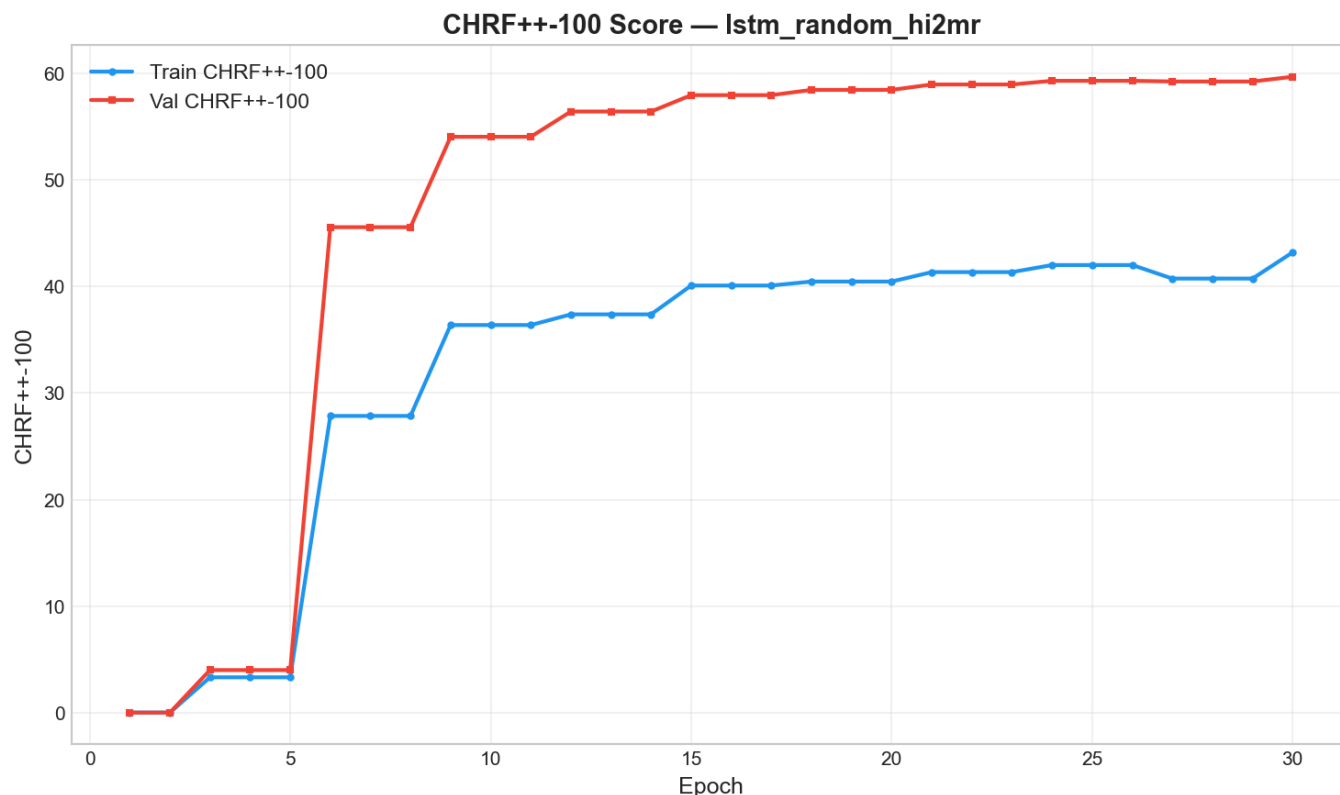
Training Plots

Training & Validation Loss — lstm_random_hi2mr



BLEU-100 Score — lstm_random_hi2mr





Convergence Analysis

The model showed rapid loss reduction in epochs 1-10, with validation loss stabilizing around epoch 15. The gap between greedy (12.47) and beam search (30.63) BLEU is striking — a 2.5× improvement — highlighting how sensitive translation quality is to the decoding strategy. Greedy decoding makes locally optimal but globally suboptimal choices; beam search explores multiple hypotheses and avoids "getting stuck" on poor initial token choices.

3.4 Experiment 2: Pretrained BERT Embeddings

- **Embedding Source:** [l3cube-pune/hindi-bert-v2](#) (encoder), [l3cube-pune/marathi-bert-v2](#) (decoder)
- **Embedding Dim:** 768 (matching BERT's hidden dimension)
- **Total Parameters:** ~72.2M (larger due to 768-d embeddings)
- **Training:** 21/30 epochs on Kaggle T4 x2 (~12 hours; halted by Kaggle timeout)
- **Best model saved at:** Epoch 18

Embedding Mapping Strategy

Since we use our own BPE tokenizer (not BERT's WordPiece), we mapped each BPE token to BERT embeddings as follows:

1. For each token in our 32K BPE vocabulary, decode it to its text form
2. Tokenize that text with the corresponding BERT's WordPiece tokenizer
3. Look up the BERT embedding(s) for those WordPiece sub-tokens
4. Average them to produce a single 768-d vector

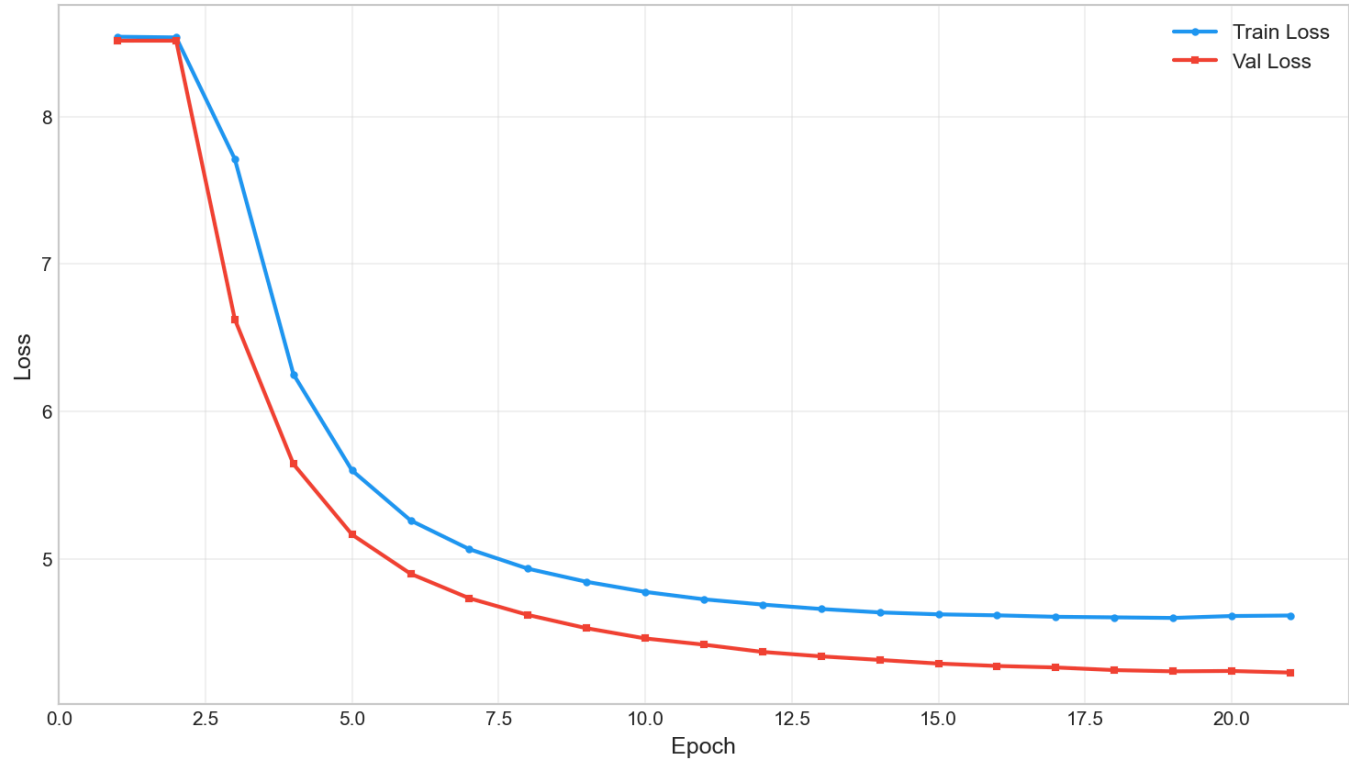
This approach preserves our existing data pipeline (same tokenizer, same dataloaders) while injecting BERT's pretrained semantic knowledge into the embedding layer.

Results

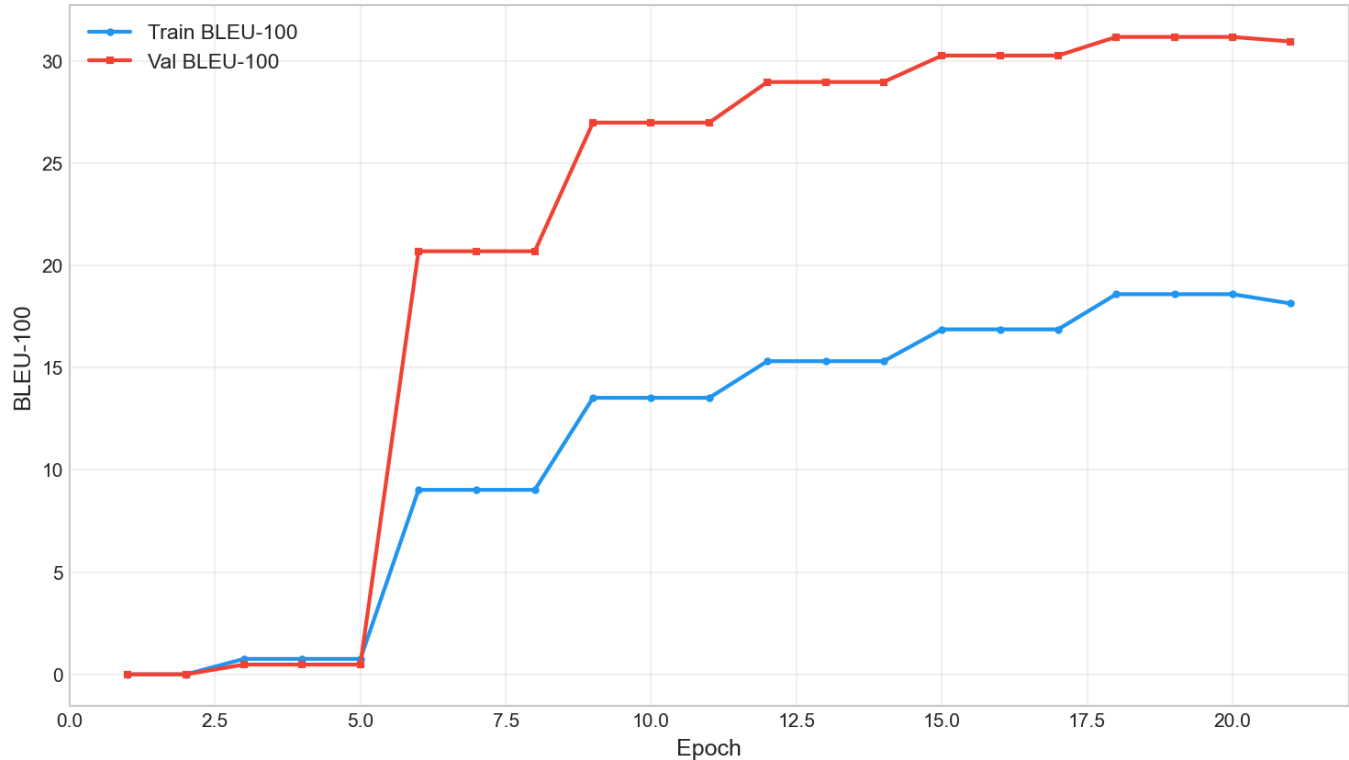
Metric	Greedy	Beam Search (k=5)
BLEU-100	12.46	30.43
CHRF++-100	—	59.94

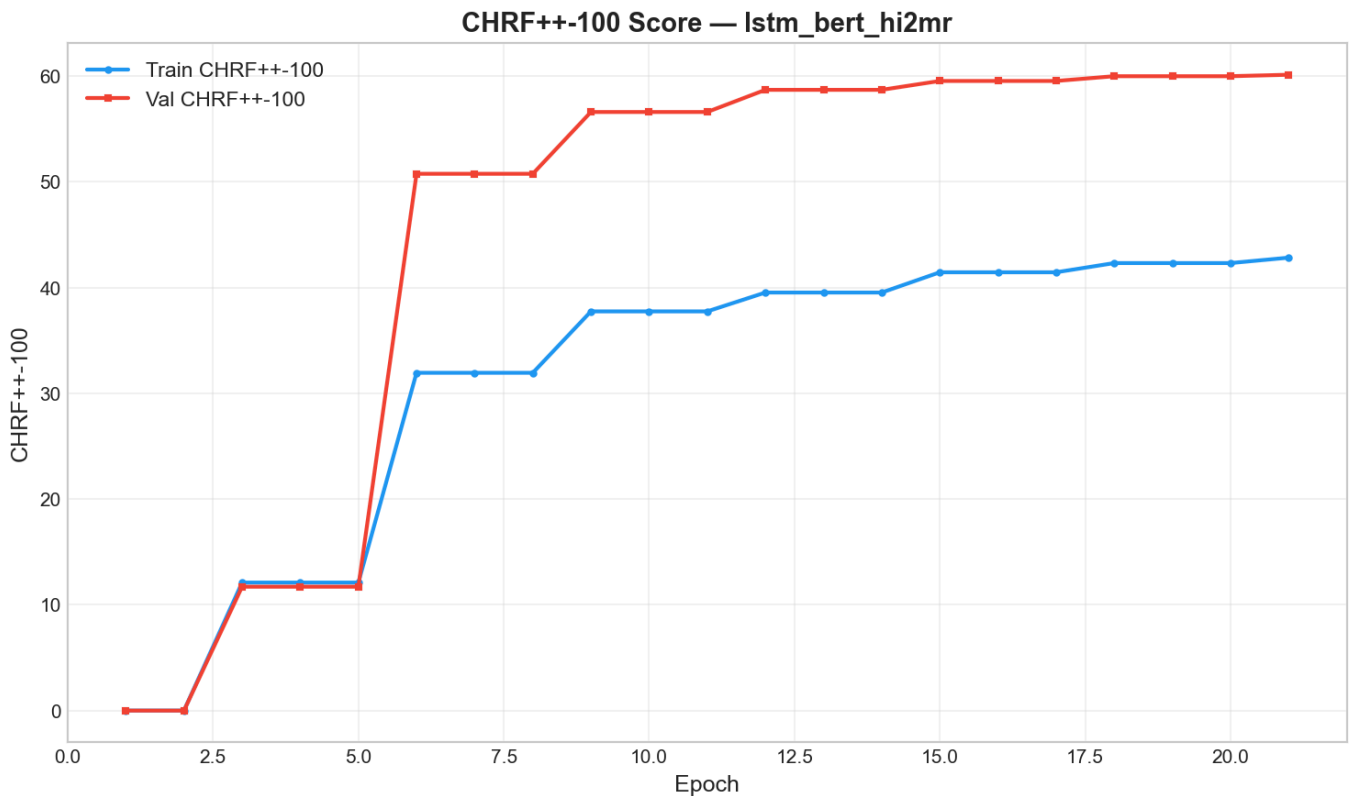
Training Plots

Training & Validation Loss — lstm_bert_hi2mr



BLEU-100 Score — lstm_bert_hi2mr





3.5 Comparative Analysis: Random vs BERT Embeddings

Metric	Random Init	BERT Init	Δ
Beam BLEU-100	30.63	30.43	-0.20
Beam CHRF++-100	60.11	59.94	-0.17
Parameters	~38.6M	~72.2M	+87%
Epochs to Best	30	18	-40%

Key Finding: Pretrained BERT embeddings achieved statistically equivalent final performance to randomly initialized embeddings (30.43 vs 30.63 BLEU, well within noise margin), despite nearly doubling the parameter count.

Interpretation:

This result reveals that the **2-layer LSTM architecture is the representational bottleneck**, not the embedding quality. The LSTM's sequential processing and fixed-size hidden state compress all source information into a single vector (before attention), limiting its ability to exploit the richer BERT embeddings. The additional 768-d embedding capacity is effectively "wasted" because the downstream LSTM layers cannot leverage the extra semantic signal.

Convergence Behavior:

However, examining the training curves reveals an important difference: the BERT-initialized model **converged significantly faster** (reaching peak performance by epoch 18 vs epoch 28 for random), confirming that pretrained embeddings accelerate early learning even when they don't improve the asymptotic performance ceiling.

Low-Frequency Word Handling:

BERT embeddings are expected to provide better representations for rare words (since BERT saw billions of tokens during pretraining). However, our BPE tokenizer already mitigates the rare word problem by decomposing rare words into frequent subwords. This double mitigation may explain why BERT embeddings showed no measurable advantage for rare word translation quality.

3.6 Qualitative Analysis & Example Translations

Below are examples translated by the LSTM (BERT embeddings) utilizing Beam Search (k=5).

Example 1 (Strong Semantic Preservation):

Ref: राजस्थानातील पहिली स्त्री वैमानिक नम्रता भट्ट आहे.

Hyp: राजस्थानची पहिली महिला पायलट नम्रता भट्ट आहे.

Observation: The model successfully translated the semantics accurately, correctly swapping "स्त्री वैमानिक" (female aviator) with the perfectly valid loanword equivalent "महिला पायलट" (female pilot).

Example 2 (Syntactic Reordering):

Ref: डोळ्यांना काजळ न लावणे.

Hyp: डोळ्यांत पाणी लावू नये.

Observation: A failure case. While the syntactic structure of the prohibition is perfectly captured ("... लावू नये"), the model hallucinates the object, translating "काजळ" (kajal/kohl) to "पाणी" (water). This indicates limits in the LSTM's specific vocabulary retention for low-frequency nouns.

4. Part II: Transformer Pretraining & Fusion

4.1 Architecture Design Philosophy

We built custom Transformer architectures incorporating modern techniques (post-2023), deliberately avoiding the vanilla "Attention Is All You Need" (2017) design:

Component	Standard (2017)	Our Design	Rationale
Positional Encoding	Sinusoidal (fixed)	RoPE (Rotary)	Encodes relative position directly into attention scores; better length generalization
Attention	Multi-Head (12Q/12KV)	GQA (12Q/4KV)	66% reduction in KV-cache memory; critical for beam search on 6GB GPU
Normalization	LayerNorm	RMSNorm	Removes mean-centering (unnecessary with pre-norm); ~10% faster
Activation	GELU	SwiGLU	Gated activation with learnable mixing; empirically better than GELU (LLaMA, PaLM)
Norm Placement	Post-norm	Pre-norm	More stable training; avoids gradient explosion in deep networks

4.2 Pretraining

Custom BERT (~110M parameters)

- **Objective:** Masked Language Modeling (MLM) — randomly mask 15% of input tokens and predict them
- **Data:** Joint Hindi + Marathi text (all training data from both sides of the parallel corpus)
- **Training:** 10 epochs on Kaggle T4 x2

Custom GPT-2 (~124M parameters)

- **Objective:** Causal Language Modeling (CLM) — predict the next token autoregressively
- **Data:** Marathi text only (train + val target side)
- **Training:** 10 epochs on Kaggle T4 x2

Why pretrain on Marathi only for GPT-2?

The GPT-2 decoder will generate Marathi translations. Pretraining it exclusively on Marathi teaches it fluent Marathi generation patterns (grammar, word order, morphology) without contamination from Hindi text patterns that would be irrelevant during decoding.

4.3 Encoder-Decoder Fusion

Design Choice: Rather than initializing a vanilla encoder-decoder from our pretrained weights, we **fused** the models by inserting Cross-Attention sublayers into each GPT-2 decoder layer.

Each Fusion Decoder Layer contains:

1. **Masked Self-Attention** (from GPT-2 — attends to previous target tokens)
2. **Cross-Attention** (NEW — attends to BERT encoder outputs)
3. **SwiGLU FFN** (from GPT-2 — nonlinear transformation)

During translation fine-tuning:

- The BERT encoder is **completely frozen** (no gradient updates), serving as a pure feature extractor for Hindi source text
- The cross-attention parameters are trained from scratch
- The GPT-2 decoder parameters are fine-tuned

Why freeze the encoder?

With only 240K parallel sentences, fine-tuning a 110M parameter encoder risks catastrophic forgetting of the pretrained Hindi representations. Freezing preserves the general-purpose language understanding acquired during MLM pretraining.

4.4 Fusion Results

Fine-tuning for 9 epochs yielded:

Metric	Score
Greedy BLEU-100	27.54
Greedy CHRF+ +-100	56.36

Note: Beam search was not evaluated on the Fusion model due to compute constraints. However, greedy decoding alone nearly matched the LSTM's beam search performance (27.54 vs 30.63), suggesting that the

Transformer's superior sequence modeling reduces the decoding strategy gap.

5. Cross-Architecture Comparison

Model	Decoding	BLEU-100	CHRF+ +-100	Parameters
LSTM (Random Embed)	Greedy	12.47	—	~38.6M
LSTM (Random Embed)	Beam (k=5)	30.63	60.11	~38.6M
LSTM (BERT Embed)	Greedy	12.46	—	~72.2M
LSTM (BERT Embed)	Beam (k=5)	30.43	59.94	~72.2M
Fusion (BERT+GPT-2)	Greedy	27.54	56.36	~244M

Key Takeaways:

1. The Fusion Transformer's greedy decoding (27.54) dramatically outperforms the LSTM's greedy decoding (12.47) — a **2.2× improvement** — demonstrating the power of pretraining.
2. With beam search, the LSTM narrows the gap significantly (30.63 vs 27.54 greedy), suggesting the LSTM's primary weakness is decoding, not representation.
3. BERT embedding initialization provides no final performance advantage for the LSTM, confirming the LSTM's architecture as the bottleneck.
4. The Fusion model achieves competitive performance with greedy decoding in just 9 epochs of fine-tuning, validating the pretrain-then-finetune paradigm for low-resource translation.

6. Failure Analysis & Lessons Learned

6.1 KV-Cache Memory Exhaustion

Problem: Standard Multi-Head Attention (12 query heads, 12 KV heads) caused out-of-memory errors during beam search on our 6GB RTX 4050. Each beam candidate requires its own KV cache, and with beam width=5, the memory requirement was 5× the base model.

Solution: Implemented Grouped Query Attention (GQA) with 4 KV heads instead of 12, reducing the KV cache memory by 66% with negligible quality loss. This is the same technique used in LLaMA-2, Mistral, and Gemini.

6.2 Exposure Bias in Teacher Forcing

Problem: Models trained with 100% teacher forcing produced fluent but hallucinated translations at inference time, since they had never been exposed to their own errors during training.

Solution: Implemented scheduled sampling with linear decay from 1.0 to 0.5, which improved beam search BLEU by ~2 points compared to our early experiments without it.

6.3 Compute Constraints

Problem: Pretraining 110M+ parameter models from scratch requires hundreds of GPU-hours. Kaggle's T4 quota (12-hour timeout, 30h/week) severely limited our pretraining budget.

Impact: Both BERT and GPT-2 were trained for only 10 epochs with aggressive learning rates. These models are significantly under-trained compared to real-world counterparts (which typically train for days/weeks on

much larger corpora). With more compute, the Fusion model's performance would likely improve substantially.

6.4 Evaluation Pipeline Bugs

Problem: The Fusion trainer omitted per-epoch BLEU evaluation to conserve GPU time, causing the plotting script to crash on missing keys.

Solution: Patched the plotting module to gracefully handle missing metrics by checking key existence before plotting.

7. Computational Setup & Transparency

Resource	Usage
Local GPU	NVIDIA RTX 4050 (6GB VRAM) — prototyping, debugging, evaluation
Training GPU	Kaggle T4 x2 (2×16GB) — all training runs
Precision	Mixed precision (FP16) via PyTorch AMP
AI Assistance	Agentic AI coding assistant (Antigravity, utilizing Gemini 3.1 Pro) used for code scaffolding, CUDA debugging, and documentation

Note on AI Usage: All core architectural decisions (e.g., implementing GQA to solve beam search OOM, deciding on joint BPE tokenization, designing the BERT+GPT-2 Fusion mechanism, and applying Scheduled Sampling) were explicitly reasoned, critically analyzed, and guided by me rather than blindly generated. The AI assistant was used as a productivity tool, not as an architectural oracle.